



Juice-shop

Vulnerability Assessment and Penetration Test Report

Tester : Jothikumar

May 11, 2026

Table of Contents

1	Statement of Confidentiality	3
2	Engagement Contacts	4
3	Executive Summary	5
3.1	Approach	5
3.2	Scope	5
3.3	Assessment Overview and Recommendations	6
4	Web Application Assessment Summary	8
4.1	Summary of Findings	8
5	Technical Findings Details	10
	Information Disclosure via API Over-fetching in Comment Section	10
	SQL Injection (SQLi) Authentication Bypass on Login Page	12
	Reflected Cross-Site Scripting (XSS) via Search Functionality	14
	Sensitive Information Disclosure via Exposed Prometheus Metrics Endpoint	16
	Business Logic Flaw	18
	Sensitive Directory Listing / Information Disclosure via Exposed FTP Path	21
	Privilege Escalation via Parameter Injection during Registration	23
	Broken Object Level Authorization (BOLA) / Parameter Impersonation via Author Overriding	25
	Business Logic Flaw / Integer Underflow via Negative Order Quantity	28
	Sensitive Information Disclosure via Exposed Server Access Log File	30
	Insecure Direct Object Reference (IDOR) / Broken Object Level Authorization (BOLA) in Basket Access	32
	Arbitrary File Download / Access Control Bypass via Null Byte Injection	34
A	Appendix	36
A.1	External Security References	36

1 Statement of Confidentiality

The contents of this document have been compiled as a result of a Vulnerability Assessment and Penetration Testing (VAPT) engagement. All architectural diagrams, discovered vulnerabilities, request/response payloads, and systemic risk classifications detailed herein are considered proprietary and strictly business confidential information.

This information is to be utilized exclusively by the authorized development, operations, and security engineering teams to remediate identified security flaws and harden the application platform. This document may not be released to external vendors, business partners, or third-party contractors without prior written consent. Additionally, no portion of this report may be communicated, reproduced, copied, or distributed across public or unsecured communication channels.

The findings and technical remediation recommendations contained in this document do not constitute formal legal or regulatory compliance advice. The assessment detailed herein was conducted against a deployment of the OWASP Juice Shop application. This environment serves as a dedicated, intentionally vulnerable web application configuration utilized strictly for security training, capability demonstration, and vulnerability mapping parameters; the documented vulnerabilities in no way affect or reflect risks within actual production corporate operations or internal network infrastructure.

2 Engagement Contacts

OWASP Contacts		
Contact	Title	Contact Email
Administrator	Admin	admin@juice-sh.op

Assessor Contact		
Assessor Name	Title	Assessor Contact Email
Jothikumar	Student	mail@mail.com

3 Executive Summary

Open Worldwide Application Security Project (“OWASP” herein) invited Jothikumar to a private engagement to perform a targeted Web Application Penetration Test of OWASP’s web application to identify high-risk security weaknesses, determine the impact to OWASP, document all findings in a clear and repeatable manner, and provide remediation recommendations.

3.1 Approach

Jothikumar performed testing under a “Black Box” approach from May 2, 2026, to May 9, 2026 without credentials or any advance knowledge of OWASP’s web application with the goal of identifying unknown weaknesses. Testing was performed from a non-evasive standpoint with the goal of uncovering as many misconfigurations and vulnerabilities as possible. Testing was performed remotely. Each weakness identified was documented and manually investigated to determine exploitation possibilities and escalation potential. Jothikumar sought to demonstrate the full impact of every vulnerability, up to and including internal network access.

3.2 Scope

The scope of this assessment was focused strictly on the OWASP Juice Shop web application architecture. This included testing the front-end user interface, backend application programming interfaces (APIs), and data handling workflows accessible via:

- **Primary Application URL:** <http://localhost:3000>
- **API Endpoints:** Any and all REST API routes ([/api/Users](#) , [/rest/products](#) , etc.) discovered via active crawling and traffic interception.
- **Network Infrastructure:** Any open web server ports (e.g., TCP ports 80, 443, 3000) or associated services hosting the application infrastructure.

Host/URL/IP Address	Description
http://localhost:3000	Primary Application URL
API Endpoints	Any and all REST API routes (/api/Users , /rest/products , etc.) discovered via active crawling and traffic interception
Network Infrastructure	Any open web server ports (e.g., TCP ports 80, 443, 3000) or associated services hosting the application infrastructure

The following types of findings were in-scope for this assessment:

- Sensitive or personally identifiable information disclosure
- Cross-Site Scripting (XSS)
- Server-side or remote code execution (RCE)
- Arbitrary file upload
- Authentication or authorization flaws, such as insecure direct object references (IDOR), and authentication bypasses

-
- All forms of injection vulnerabilities
 - Directory traversal
 - Local file read
 - Significant security misconfigurations and business logic flaws
 - Exposed credentials that could be leveraged to gain further access

The following types of activities were considered out-of-scope for this assessment:

- Scanning and assessing any other IP in the Entry Point's network
- Physical attacks against OWASP properties
- Unverified scanner output
- Man-in-the-Middle attacks
- Any vulnerabilities identified through DDoS or spam attacks
- Self-XSS
- Login/logout CSRF
- Issues with SSL certificates, open ports, TLS versions, or missing HTTP response headers
- Vulnerabilities in third party libraries unless they can be leveraged to significantly impact the target
- Any theoretical attacks or attacks that require significant user interaction or low risk

3.3 Assessment Overview and Recommendations

1. Assessment Overview

This security assessment focused on evaluating the runtime security posture, logical access control boundaries, and input validation resilience of the target application.

The engagement uncovered multiple systemic vulnerabilities ranging from Medium to Critical severity. The core security weaknesses identified point toward a systemic reliance on client-side security mechanisms, missing server-side authorization controls (BOLA/IDOR), and inadequate neutralization of structured query logic and special characters (SQLi, XSS, and Null Byte Injection).

Left unaddressed, these flaws collectively allow unauthenticated users to bypass login screens, escalate privileges to full platform administrators, view or manipulate other users' proprietary checkout baskets, and access internal system logs or configuration archives.

2. Strategic Root-Cause Analysis

The vulnerabilities discovered during this assessment are not isolated operational errors; rather, they stem from three core engineering anti-patterns:

1. **Implicit Trust in Client Parameters (Client-Side Bias):** Vulnerabilities such as Mass Assignment, Parameter Impersonation, and Business Logic Flaws (Negative/Zero values) occur because the backend framework directly accepts, processes, or trusts data structures supplied by the frontend interface. The architecture treats the browser as a secure validation environment rather than a mutable presentation layer.
2. **Missing Object-Level Authorization Logic:** The application repeatedly fails to map active session identities (tokens) against the exact data records being mutated or viewed. Simply checking if a user is "logged in" is insufficient; the backend lacks data-layer validation to prove whether the requesting user actually owns the target basket or product review entry.

3. **Insecure Infrastructure Boundaries and Perimeter Controls:** Administrative utilities, system monitoring tools (`/metrics`), and transaction archives (`access.log` , directory listings) are exposed due to a lack of reverse proxy access restrictions and perimeter routing blocks. Production environments are deploying raw diagnostics to folders within the public asset web root.

3. Executive Recommendations

To comprehensively resolve these issues and establish a resilient baseline against future exploitation, the development and infrastructure teams should prioritize the following programmatic remedies:

Phase 1: Immediate Structural Remediation (High-Priority)

- **Implement Prepared Statements Universally:** Mandate the use of parameterized inputs across all database connectors to entirely neutralize SQL Injection anomalies.
- **Enforce Strict Object-Level Access Verification:** Ensure that whenever a resource (such as a basket or review author field) is accessed or altered, the server verifies that the unique identifier embedded in the cryptographically signed session token matches the owner of that database record.
- **Adopt Explicit Data Transfer Objects (DTOs):** Prevent Mass Assignment vulnerabilities by creating strict request binding schemas (e.g., Zod, Joi) that drop un-whitelisted parameters like `role: admin` or out-of-bounds metrics like negative integers.

Phase 2: Ingress and Perimeter Hardening

- **Restrict Access to Administrative Endpoints:** Restrict paths like `/metrics` and `/logs` via the reverse proxy (e.g., Nginx, Ingress) or network-level firewall access control lists (ACLs) so they are completely inaccessible from the public internet.
- **Sanitize Inputs for Null Elements and Special Characters:** Implement a global request parser interceptor that searches for and blocks URL-encoded null bytes (`%00`, `%2500`) and standard cross-site scripting markers, while applying context-aware output encoding to all user content reflected back to the browser.

4 Web Application Assessment Summary

Jothikumar began all testing activities from the perspective of an unauthenticated user on the internet. OWASP provided the tester with a single URL and IP address but did not provide additional information such as operating system or configuration information.

4.1 Summary of Findings

During the course of testing, Jothikumar uncovered a total of 12 findings that pose a material risk to OWASP's information systems. The below chart provides a summary of the findings by severity level.

In the course of this penetration test **2 Critical**, **5 High** and **5 Medium** vulnerabilities were identified:

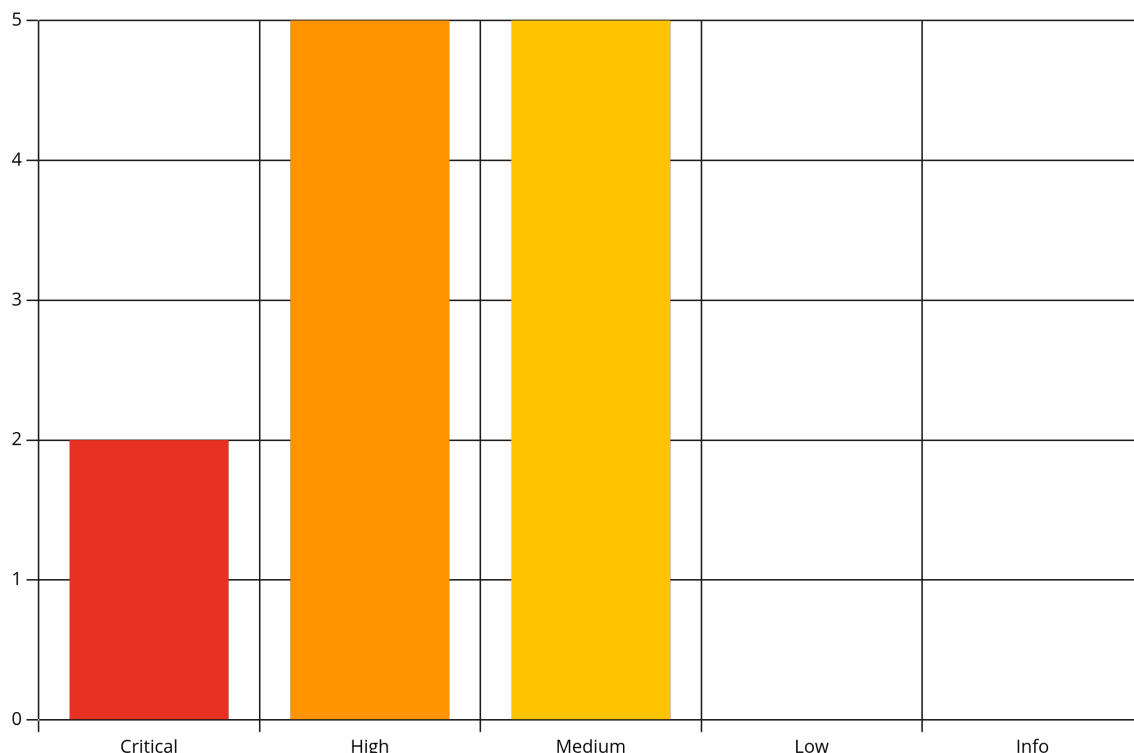


Figure 1 - Distribution of identified vulnerabilities

Below is a high-level overview of each finding identified during testing. These findings are covered in depth in the Technical Findings Details section of this report.

#	Severity Level	Finding Name	Page
1	7.5 (High)	Information Disclosure via API Over-fetching in Comment Section	10
2	9.8 (Critical)	SQL Injection (SQLi) Authentication Bypass on Login Page	12
3	8.2 (High)	Reflected Cross-Site Scripting (XSS) via Search Functionality	14
4	5.3 (Medium)	Sensitive Information Disclosure via Exposed Prometheus Metrics Endpoint	16

#	Severity Level	Finding Name	Page
5	5.3 (Medium)	Business Logic Flaw	18
6	5.3 (Medium)	Sensitive Directory Listing / Information Disclosure via Exposed FTP Path	21
7	9.8 (Critical)	Privilege Escalation via Parameter Injection during Registration	23
8	6.5 (Medium)	Broken Object Level Authorization (BOLA) / Parameter Impersonation via Author Overriding	25
9	7.1 (High)	Business Logic Flaw / Integer Underflow via Negative Order Quantity	28
10	7.5 (High)	Sensitive Information Disclosure via Exposed Server Access Log File	30
11	6.5 (Medium)	Insecure Direct Object Reference (IDOR) / Broken Object Level Authorization (BOLA) in Basket Access	32
12	7.5 (High)	Arbitrary File Download / Access Control Bypass via Null Byte Injection	34

5 Technical Findings Details

1. Information Disclosure via API Over-fetching in Comment Section - High

CWE	CWE-200 - Exposure of Sensitive Information to an Unauthorized Actor
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Owasp Category	A01:2025 - Broken Access Control
Root Cause	The application suffers from an Information Disclosure vulnerability within the comment section. When a user navigates to a page containing comments, the frontend requests the comment data from the backend API. However, the backend API fails to filter the returned data. Instead of solely returning the required public fields, it returns the entire user object associated with the comment. This exposes sensitive metadata, such as user email addresses, internal database IDs, password hashes, or account role flags, to any user monitoring the network traffic.
Impact	- Harvest sensitive Personally Identifiable Information (PII) like email addresses, leading to targeted phishing or credential stuffing attacks. - Enumerate valid user accounts on the platform.
Affected Component	/rest/products/1/reviews
Remediation	The application backend should strictly enforce the Principle of Least Privilege regarding data transmission. The API should only send the exact data required by the frontend client to render the view. Implement Data Transfer Objects (DTOs): Do not pass raw database models directly to the HTTP response formatter. Create specific DTOs that only contain safe, public properties Explicit Field Selection: Update the database query to exclusively select the necessary columns. Data Masking: If certain identifiers must be sent for application state logic, ensure they are masked, encrypted, or replaced with non-sequential, opaque UUIDs rather than sequential internal database IDs.
References	- OWASP Top 10 Mapping: A01:2025 - Broken Access Control (Under the expanded scope of unauthorized data access/over-fetching across API endpoints) [https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/] - OWASP Cheat Sheet Series: Mass Assignment & Over-fetching Prevention [https://cheatsheetseries.owasp.org/] - MITRE CWE: CWE-200: Exposure of Sensitive Information to an Unauthorized Actor [https://cwe.mitre.org/data/definitions/200.html]



Finding Evidence

[illegible]

2. SQL Injection (SQLi) Authentication Bypass on Login Page

- Critical

CWE	CWE-89 - Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection')
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Owasp Category	A05:2025 - Injection
Root Cause	The application's login authentication module is vulnerable to a classic SQL Injection (SQLi) flaw. When a user submits their login credentials, the input provided in the email field is concatenated directly into a backend database query statement rather than using parameterized inputs or prepared statements. By entering characters that break the SQL syntax (such as '), an attacker can manipulate the query logic. Injecting an expression that evaluates to TRUE (e.g., ' OR 1=1--) trick the database query into validating successfully without requiring a legitimate password. Furthermore, because database structures often return the first available index record when universal parameters match, an attacker can intentionally target and access the administrative user account or any arbitrary user identity.
Impact	• This is a high-consequence vulnerability that completely compromises the data plane and user authentication controls of the application. An unauthenticated attacker can exploit this flaw to: • Bypass authentication checks entirely and access administrative panels. • Impersonate any registered customer or executive entity, accessing sensitive Personally Identifiable Information (PII) and corporate accounts. • Exfiltrate, drop, or alter database content depending on the access rights assigned to the web application's database connection.
Affected Component	/rest/user/login
Remediation	• To permanently resolve structural injection capabilities, user inputs must never modify the intent or logical syntax of an active interpretation language query. • Implement Parameterized Queries / Prepared Statements: Utilize your backend database driver interface or an Object-Relational Mapper (ORM) to handle variables natively via typed boundaries. • Enforce Strict Input Sanitization: Reject values that contain pattern matching components inconsistent with structured format schemas (such as alphanumeric restrictions on profile fields). • For detailed information and assistance on how to prevent SQL Injection vulnerabilities, see OWASP's linked SQL Injection Prevention Cheat Sheet.
References	• OWASP Top 10 Mapping: A05:2025 - Injection [https://owasp.org/Top10/2025/A05_2025-Injection/] • OWASP Cheat Sheet Series: SQL Injection Prevention Cheat Sheet [https://cheatsheetseries.owasp.org/cheatsheets/SQL_Injection_Prevention_Cheat_Sheet.html]

OWASP

- MITRE CWE: CWE-89: Improper Neutralization of Special Elements used in an SQL Command ('SQL Injection') [<https://cwe.mitre.org/data/definitions/89.html>]

Finding Evidence

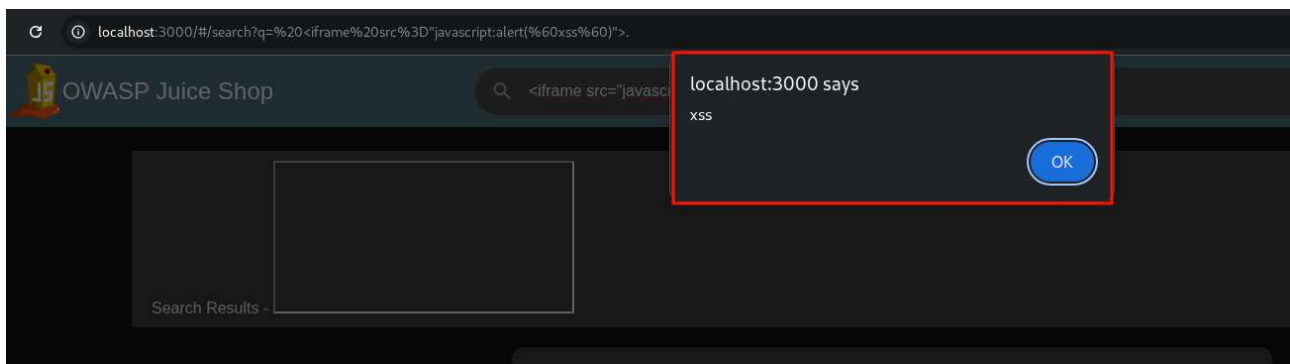
[illegible]

3. Reflected Cross-Site Scripting (XSS) via Search Functionality - High

CWE	-
CVSS 3.1	8.2 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:L/A:N
Owasp Category	A05:2025 - Injection
Root Cause	The application's global search functionality is vulnerable to Reflected Cross-Site Scripting (XSS). When a user submits a search query, the input string is reflected directly back into the server's HTML response without undergoing proper context-aware output encoding or sanitization. By inserting an input containing structured HTML tags, an attacker can manipulate the Document Object Model (DOM) rendered by the victim's web browser. In this instance, supplying an <iframe> tag configured with a JavaScript pseudo-protocol execution source (src="javascript:...") forces the browser to open an embedded inline frame and immediately execute arbitrary JavaScript commands within the context of the user's current web session.
Impact	• Because the malicious script executes inside the victim's browser session, an attacker can leverage this flaw to perform unauthorized actions on behalf of the user. Potential impacts include: • Session Hijacking: Stealing session cookies or web tokens (if HttpOnly flags are absent), allowing full account takeover. • Defacement/Phishing: Dynamically altering the web page content to display fraudulent login prompts or misleading information. • Forced Actions: Silently triggering state-changing requests, such as updating account profiles, changing passwords, or initiating background transactions.
Affected Component	/rest/products/search?q=

Remediation	To permanently prevent Cross-Site Scripting vulnerabilities, the application must treat all user-supplied data as untrusted literals rather than executable data streams. • Implement Context-Aware Output Encoding: Ensure that any user-controlled input reflected back into the HTML body is strictly encoded before rendering. Convert dynamic characters into safe HTML entities (e.g., replace < with <, > with >). • Deploy a Content Security Policy (CSP): Enforce strict browser execution boundaries via a defense-in-depth header mechanism. Restrict the usage of child-src/frame-src bindings and disable risky evaluation privileges. • Example HTTP Header configuration: Content-Security-Policy: default-src 'self'; frame-src 'none'; • Utilize Modern UI Framework Protections: If using modern libraries (like Angular, React, or Vue), leverage their native context-safe data binding systems (e.g., double curly braces {{ syntax) while explicitly avoiding bypass methods like dangerouslySetInnerHTML
References	• OWASP Top 10 Mapping: A05:2025 - Injection (XSS vulnerabilities fall directly within the broader scope of Injection risks) [https://owasp.org/Top10/2025/] • OWASP Cheat Sheet Series: Cross-Site Scripting Prevention Cheat Sheet [https://cheatsheetseries.owasp.org/cheatsheets/Cross_Site_Scripting_Prevention_Cheat_Sheet.html] • MITRE CWE: CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')[https://cwe.mitre.org/data/definitions/79.html]

Finding Evidence

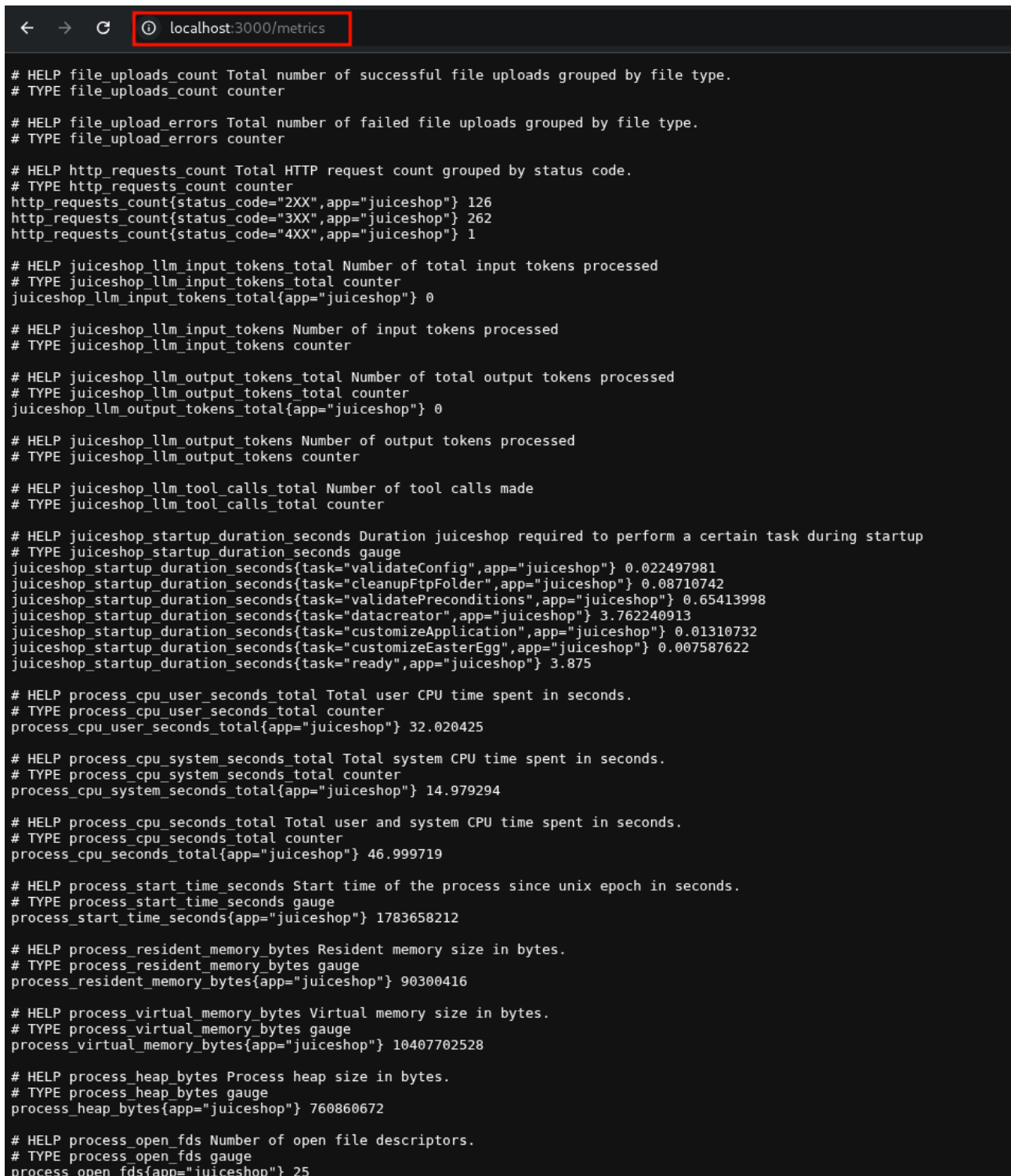


4. Sensitive Information Disclosure via Exposed Prometheus Metrics Endpoint - Medium

CWE	CWE-202 - Exposure of Sensitive Information Through Data Queries
CVSS 3.1	5.3 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Owasp Category	A01:2025 - Broken Access Control
Root Cause	The application configuration exposes a public-facing infrastructure monitoring endpoint (<code>/metrics</code>) designed for the Prometheus scraper. This deployment lacks IP whitelisting, perimeter routing blocks, or active session authorization checks. When queried, the endpoint exports verbose, real-time environment telemetry data. This exposure leaks comprehensive details regarding internal system architecture, including system memory structures, CPU load thresholds, active database connections, framework specific garbage collection variables, and internal naming layouts or routing rules.
Impact	While an exposed metrics endpoint does not grant an attacker direct remote command execution or file read capabilities, it functions as a highly effective reconnaissance source. An unauthenticated attacker can leverage this exposure to: • Map out infrastructure blueprints and identify precise server/runtime frame technologies. • Target memory leak points or track thread spikes to orchestrate precision Denial of Service (DoS) conditions. • Leak business-sensitive indicators implicitly, such as tracking processing throughput or monitoring traffic spikes.
Affected Component	<code>/metrics</code>
Remediation	System operational parameters and telemetry data are internal technical data points that should never intersect with the public Internet perimeter. • Apply Network Access Control Lists (ACLs): Enforce internal network boundaries via a reverse proxy (such as Nginx, AWS CloudFront, or an ingress controller) to block access requests to <code>/metrics</code> originating from outside the trusted enterprise network interface. • Enforce Authentication Mechanisms: If external network components must grab data from this path over open web routes, secure the route using a strong API Key, Mutual TLS (mTLS), or Basic Authentication protocols supported by the Prometheus scraper engine. • Disable Production Endpoint Mappings: If the environment metrics monitoring logic is redundant or handled locally, explicitly toggle the scraper endpoint exposure off in production environment files.
References	• OWASP Top 10 Mapping: A01:2025 - Broken Access Control (Improper route restrictions granting unauthenticated access to internal metrics utilities) [https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/]

- OWASP Cheat Sheet Series: Denial of Service Prevention Cheat Sheet [<https://cheatsheetseries.owasp.org/>]
- MITRE CWE: CWE-200: Exposure of Sensitive Information to an Unauthorized Actor [<https://cwe.mitre.org/data/definitions/200.html>]

Finding Evidence



```
localhost:3000/metrics

# HELP file_uploads_count Total number of successful file uploads grouped by file type.
# TYPE file_uploads_count counter

# HELP file_upload_errors Total number of failed file uploads grouped by file type.
# TYPE file_upload_errors counter

# HELP http_requests_count Total HTTP request count grouped by status code.
# TYPE http_requests_count counter
http_requests_count{status_code="2XX",app="juiceshop"} 126
http_requests_count{status_code="3XX",app="juiceshop"} 262
http_requests_count{status_code="4XX",app="juiceshop"} 1

# HELP juiceshop_llm_input_tokens_total Number of total input tokens processed
# TYPE juiceshop_llm_input_tokens_total counter
juiceshop_llm_input_tokens_total{app="juiceshop"} 0

# HELP juiceshop_llm_input_tokens Number of input tokens processed
# TYPE juiceshop_llm_input_tokens counter

# HELP juiceshop_llm_output_tokens_total Number of total output tokens processed
# TYPE juiceshop_llm_output_tokens_total counter
juiceshop_llm_output_tokens_total{app="juiceshop"} 0

# HELP juiceshop_llm_output_tokens Number of output tokens processed
# TYPE juiceshop_llm_output_tokens counter

# HELP juiceshop_llm_tool_calls_total Number of tool calls made
# TYPE juiceshop_llm_tool_calls_total counter

# HELP juiceshop_startup_duration_seconds Duration juiceshop required to perform a certain task during startup
# TYPE juiceshop_startup_duration_seconds gauge
juiceshop_startup_duration_seconds{task="validateConfig",app="juiceshop"} 0.022497981
juiceshop_startup_duration_seconds{task="cleanupFtpFolder",app="juiceshop"} 0.08710742
juiceshop_startup_duration_seconds{task="validatePreconditions",app="juiceshop"} 0.65413998
juiceshop_startup_duration_seconds{task="datacreator",app="juiceshop"} 3.762240913
juiceshop_startup_duration_seconds{task="customizeApplication",app="juiceshop"} 0.01310732
juiceshop_startup_duration_seconds{task="customizeEasterEgg",app="juiceshop"} 0.007587622
juiceshop_startup_duration_seconds{task="ready",app="juiceshop"} 3.875

# HELP process_cpu_user_seconds_total Total user CPU time spent in seconds.
# TYPE process_cpu_user_seconds_total counter
process_cpu_user_seconds_total{app="juiceshop"} 32.020425

# HELP process_cpu_system_seconds_total Total system CPU time spent in seconds.
# TYPE process_cpu_system_seconds_total counter
process_cpu_system_seconds_total{app="juiceshop"} 14.979294

# HELP process_cpu_seconds_total Total user and system CPU time spent in seconds.
# TYPE process_cpu_seconds_total counter
process_cpu_seconds_total{app="juiceshop"} 46.999719

# HELP process_start_time_seconds Start time of the process since unix epoch in seconds.
# TYPE process_start_time_seconds gauge
process_start_time_seconds{app="juiceshop"} 1783658212

# HELP process_resident_memory_bytes Resident memory size in bytes.
# TYPE process_resident_memory_bytes gauge
process_resident_memory_bytes{app="juiceshop"} 90300416

# HELP process_virtual_memory_bytes Virtual memory size in bytes.
# TYPE process_virtual_memory_bytes gauge
process_virtual_memory_bytes{app="juiceshop"} 10407702528

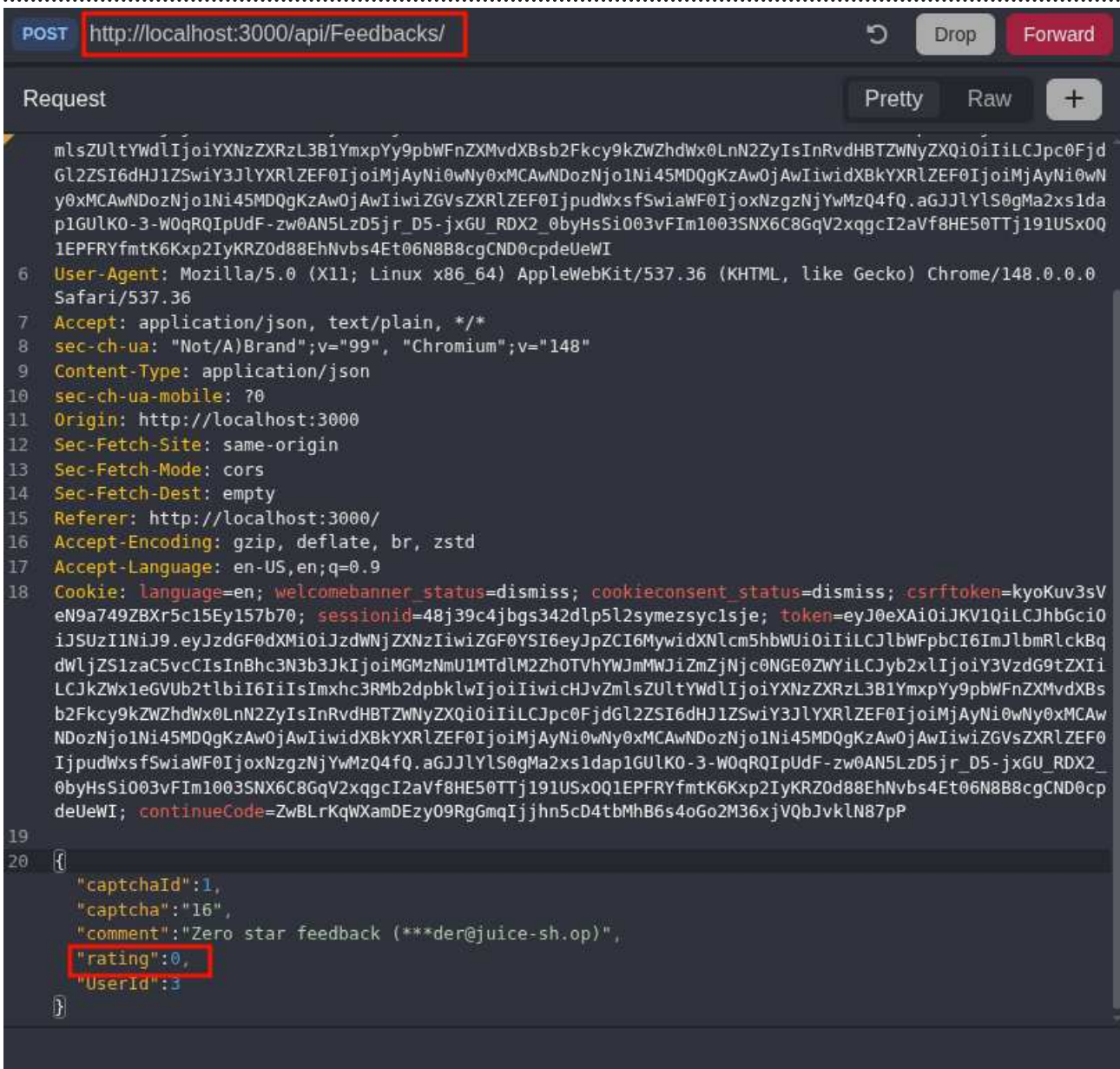
# HELP process_heap_bytes Process heap size in bytes.
# TYPE process_heap_bytes gauge
process_heap_bytes{app="juiceshop"} 760860672

# HELP process_open_fds Number of open file descriptors.
# TYPE process_open_fds gauge
process_open_fds{app="juiceshop"} 25
```

5. Business Logic Flaw - Medium

CWE	CWE-20 - Improper Input Validation
CVSS 3.1	5.3 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:N/I:L/A:N
Owasp Category	A05:2025 - Injection
Root Cause	The application features a customer feedback module where users can submit a feedback. While the frontend user interface enforces a strict selection rule restricting reviews to a specific integer boundary (e.g., a minimum of 1 star and a maximum of 5 stars), the backend server fails to validate the value submitted in the API request payload. By intercepting the legitimate request via an intercepting proxy web tool, an attacker can modify the <code>rating</code> parameter manually to a value outside the intended schema bounds—such as <code>0</code> or negative integers. Because the server stores the input directly without server-side validation checks, an attacker can alter data integrity parameters or manipulate systemic calculation scores.
Impact	Client-side user controls must be viewed strictly as interface helpers. The backend server must remain the single source of truth for validation boundaries. An attacker can exploit this flaw to: • Submit structurally invalid ratings that distort aggregate review data, potentially manipulating overall score calculations downwards. • Bypass frontend application logic boundaries, proving a lack of uniform server-side enforcement controls.
Affected Component	/api/Feedbacks/
Remediation	Client-side user controls must be viewed strictly as interface helpers. The backend server must remain the single source of truth for validation boundaries. • Enforce Strict Server-Side Validation Schema: Implement robust input verification handlers (such as Joi, Zod, or native validation libraries) that reject inputs failing to match expected boundaries. • Reject Out-of-Bounds Submissions: If an incoming API parameter violates predefined ranges, terminate the operation immediately and issue an HTTP <code>400 Bad Request</code> code.
References	• OWASP Top 10 Mapping: A05:2025 - Injection (Improper validation of data parameters falls under general input processing and injection risk vectors) [https://owasp.org/Top10/2025/A05_2025-Injection/] • OWASP Cheat Sheet Series: Input Validation Cheat Sheet [https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html] • MITRE CWE: CWE-20: Improper Input Validation [https://cwe.mitre.org/data/definitions/20.html]

Finding Evidence



```
201 http://localhost:3000/api/Feedbacks/ Drop Forward

Response Pretty Raw +

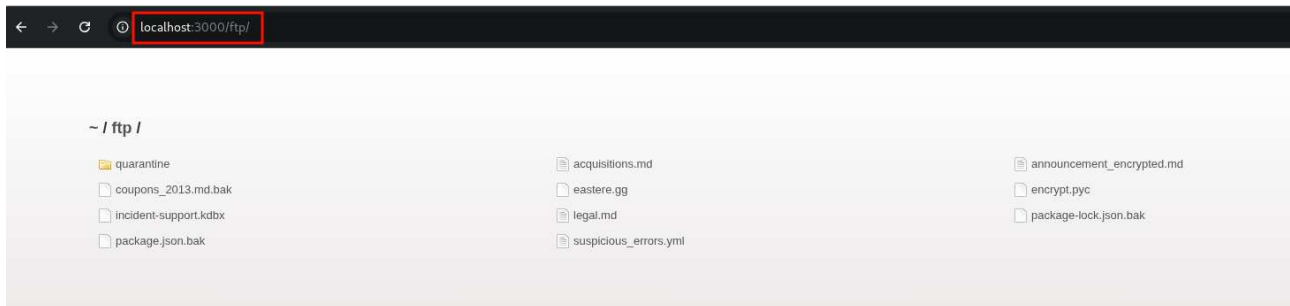
1 HTTP/1.1 201 Created
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Location: /api/Feedbacks/10
8 Content-Type: application/json; charset=utf-8
9 Content-Length: 189
10 ETag: W/"bd-74xj+v10AfSdb8gLFSNV2smW6RU"
11 Vary: Accept-Encoding
12 Date: Fri, 10 Jul 2026 05:53:29 GMT
13 Connection: keep-alive
14 Keep-Alive: timeout=5
15
16 {
  "status": "success"
  "data": {
    "id": 10,
    "comment": "Zero star feedback (**der@juice-sh.op)",
    "rating": 0,
    "UserId": 3,
    "updatedAt": "2026-07-10T05:53:29.132Z",
    "createdAt": "2026-07-10T05:53:29.132Z"
  }
}
```

6. Sensitive Directory Listing / Information Disclosure via Exposed FTP Path - Medium

CWE	CWE-548 - Exposure of Information Through Directory Listing
CVSS 3.1	5.3 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:L/I:N/A:N
Owasp Category	A01:2025 - Broken Access Control
Root Cause	The application handles file transfers or structural static files through an exposed <code>/ftp/</code> web directory route. While individual assets like <code>legal.md</code> are intended to be publicly accessible, the web server or application router fails to restrict directory listing permissions on the parent directory. When an attacker accesses the base path (<code>/ftp/</code>) by removing the explicit file name from the URL, the application returns an index list rendering every folder, configuration script, backup archive, or markdown file stored within that physical directory partition. This lack of access management exposes files that were never meant to be publicly visible or linked on the web interface.
Impact	Allowing unauthenticated directory listing introduces significant structural information disclosure risks. An attacker can exploit this flaw to: - Enumerate and harvest internal administrative notes, sensitive corporate backups, or hidden configuration data stored in the folder. - Discover forgotten or non-linked developer test files that may contain secondary design vulnerabilities or hardcoded variables. - Perform mass download automation of critical assets without interacting with standard front-end controls.
Affected Component	<code>/ftp/</code>
Remediation	Web directories holding public application assets must be configured explicitly to serve defined objects only, strictly denying automatic index creation behaviors. Disable Directory Browsing on Server/Router: Adjust your web server configuration framework or application middleware options to explicitly disable index listings. <i>Example (Express.js static middleware):</i> Ensure option parameters like <code>dotfile</code> <code>s</code> are restricted and avoid using external directory index packages in production setups. Return Authorization and Status Barriers: Configure the root path handler to return an explicit HTTP <code>403 Forbidden</code> error status page if a user requests a direct folder map without specifying a destination file entity. Implement Index Placeholders: Drop an empty index file (e.g., an empty <code>index.html</code>) directly into asset folders like <code>/ftp/</code> as a basic fallback strategy. This forces the engine to serve a blank template rather than auto-generating an information-rich list view.
References	

- OWASP Top 10 Mapping: A01:2025 - Broken Access Control (Improper path restriction leading to exposure of internal file directory configurations)[https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/]
- OWASP Cheat Sheet Series: Secure Cloud Architecture Cheat Sheet [<https://cheatsheetseries.owasp.org/>]
- MITRE CWE: CWE-548: Exposure of Information Through Directory Listing [<https://cwe.mitre.org/data/definitions/548.html>]

Finding Evidence

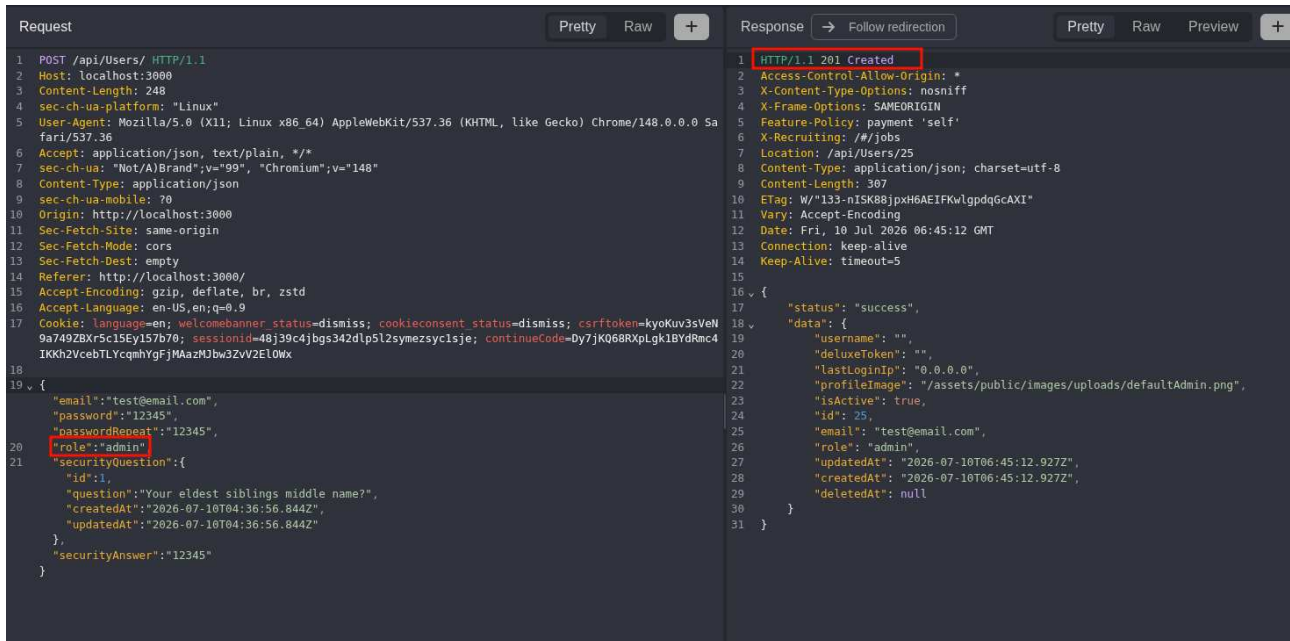


7. Privilege Escalation via Parameter Injection during Registration - **Critical**

CWE	CWE-915 - Improperly Controlled Modification of Dynamically-Determined Object Attributes
CVSS 3.1	9.8 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H
Owasp Category	A01:2025 - Broken Access Control
Root Cause	The application's user registration functionality suffers from a Mass Assignment vulnerability. When an unauthenticated visitor fills out the registration form, the frontend submits a JSON payload to the signup endpoint. However, the backend framework binds the incoming JSON keys directly to the database object model properties without applying an explicit field whitelist. This lack of data filtering allows an attacker to inject internal metadata attributes—such as adding a <code>"role": "admin"</code> key-value pair directly into the request body. Because the object mapper blindly trusts and commits the user-provided attributes, the registration routine automatically provisions the new profile with administrative privileges.
Impact	This flaw bypasses all core access controls and security logic of the platform. An unauthenticated external entity can exploit this vulnerability to: • Instantly create fully privileged administrative accounts at will. • Exfiltrate, modify, or completely delete corporate production databases. • Manipulate sensitive system components, view hidden financial logs, or impersonate application administrators, leading to total operational compromise.
Affected Component	/api/Users/
Remediation	Applications must explicitly decouple the object models used to accept data over the network from the business data models committed to the storage layer. • Implement Strict Data Transfer Objects (DTOs): Define explicit schemas or objects that only expose parameters safe for user configuration (e.g., <code>UserSignupDTO</code> containing <i>only</i> email, password, and security answer). • Enforce Strict Property Whitelisting: If your backend language mapper binds data dynamically, explicitly configure it to permit safe parameters exclusively while dropping unauthorized input keys. • Enforce Out-of-Band Role Assignment: Ensure authorization profiles and elevated permissions cannot be passed as client parameters during public runtime interactions, handling them strictly via administrative account configuration panels.
References	• OWASP Top 10 Mapping: A01:2025 - Broken Access Control (Privilege escalation stemming from insecure parameter exposure) [https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/]

- OWASP Cheat Sheet Series: Mass Assignment Cheat Sheet [https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html]
- MITRE CWE: CWE-915: Improperly Controlled Modification of Dynamically-Determined Object Attributes [<https://cwe.mitre.org/data/definitions/915.html>]

Finding Evidence



The screenshot displays the Request and Response tabs of a web browser's developer tools. The Request tab shows a POST request to `/api/Users/` with a JSON body. The Response tab shows the resulting 201 Created status with a JSON body.

Request Body:

```
{  "email": "test@email.com",  "password": "12345",  "passwordRepeat": "12345",  "role": "admin",  "securityQuestion": {    "id": 1,    "question": "Your eldest siblings middle name?",    "createdAt": "2026-07-10T04:36:56.844Z",    "updatedAt": "2026-07-10T04:36:56.844Z"  },  "securityAnswer": "12345"}
```

Response Body:

```
{  "status": "success",  "data": {    "username": "",    "deluxeToken": "",    "lastLoginIp": "0.0.0.0",    "profileImage": "/assets/public/images/uploads/defaultAdmin.png",    "isActive": true,    "id": 25,    "email": "test@email.com",    "role": "admin",    "updatedAt": "2026-07-10T06:45:12.927Z",    "createdAt": "2026-07-10T06:45:12.927Z",    "deletedAt": null  }}}
```


8. Broken Object Level Authorization (BOLA) / Parameter Impersonation via Author Overriding - Medium

CWE	CWE-639 - Authorization Bypass Through User-Controlled Key
CVSS 3.1	6.5 / CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:N
Owasp Category	A01:2025 - Broken Access Control
Root Cause	The application's product reviews API endpoint suffers from a Broken Object Level Authorization (BOLA) flaw, often referred to as an Insecure Direct Object Reference (IDOR) within the request context. When an authenticated user submits a product review, the backend application relies on a user-supplied parameter (<code>author</code> email or ID) inside the JSON request payload to determine ownership of the post rather than extracting and verifying identity directly from the active authentication session token (JWT or Cookie). By altering the value of the <code>author</code> field via an intercepting proxy web tool, an authenticated user can submit feedback that appears to originate from any arbitrary account identity, including system administrators or corporate profiles, bypassing identity verification checks
Impact	Allowing parameter-based identity manipulation fundamentally compromises data authenticity, attribution controls, and user trust across the platform. An attacker can exploit this vulnerability to: • Spoof authoritative configurations or publish fraudulent, high-reputation product evaluations under the guise of an administrator. • Orchestrate target defamation campaigns against specific vendors or items by forging reviews from real corporate stakeholders. • Tarnish data integrity parameters across backend reporting metrics by decoupling real identity parameters from user contributions.
Affected Component	<code>/rest/products/24/reviews</code>
Remediation	Applications must never trust parameter data points supplied within incoming payloads when handling data attribution or identity context mapping. • Determine Identity on the Server: Strip the mutable identity string from the incoming API schema completely. The backend application routine must determine authorship context implicitly by querying the verified user identity claims parsed securely from the incoming server-side session token framework. • Verify Transaction Authorization Controls: Enforce robust validation layers ensuring that any explicit ID configuration parameter submitted via a client route context accurately mirrors the specific entity authorized within the request bearer properties before execution blocks proceed.
References	• OWASP Top 10 Mapping: A01:2025 - Broken Access Control (Flaws categorized under broken object-level authorization bounds) [https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/]

- [illegible]

Finding Evidence

```
PUT http://localhost:3000/rest/products/24/reviews
Request Pretty Raw +
1 PUT /rest/products/24/reviews HTTP/1.1
2 Host: localhost:3000
3 Content-Length: 70
4 sec-ch-ua-platform: "Linux"
5 Authorization: Bearer eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdGF0dXMiOiJzdWljZXNziIiwiaWF0YSI6eyJpZC
I6MjUsInVzZXJyZWllIjoiiwiZWlhaWwioiJ0ZXNQGVtYWlsLmNvbSIiInBhc3N3b3JkIjoiodDI3Y2NiMGVlYThhNzA2YzRjMzR
hMTY4OTFmODRLN2IiLCJyb2xlbGljoiYWRtaW4iLCJkZmxleGVub2t1biI6IiIsImxhc3Rmb2dpbkklwiMC4wLjAuMCIiInByb2Zp
bGVjbWFnZSI6Ii9hc3NldHMvchVibGllL2ltYwdlcY9lCGxvYWRzL2RlZmFlbHRBZG1pb5wbmcilCJ0b3RwU2VjcV0iIjoiiwia
XNB3Y3RpdMU0OnRydWUsImNyZWFiZWRRBdCi6IjIwMjYtMDctMTAgMDY6NDU6MTIuOTI3ICswMDowMCIiInVwZGF0ZWRBdCI6IjIwMj
YtMDctMTAgMDY6NDU6MTIuOTI3ICswMDowMCIiImRlbGV0ZWRBdCI6bnVsbnVhdH0sImRhbmQzMzY2NjQ2M30.psv3JMgOB2pRKIK
3sZG8R0BenbwttUrmIFWqBJhpSRPJ6SFNKCMKdcGLI2mdkQBec5_brktOJIGMJ88fzLS03PDQ7gEsNkvPnor5wjaioUcmsjresJUi
IZFu50nVaREyDOKNEsX8X8PVx5Kfk9Lm9xyQ9gL72gsC6mk9ayKKRsM
6 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/148.0.0.0 S
afari/537.36
7 Accept: application/json, text/plain, */*
8 sec-ch-ua: "Not/A)Brand";v="99", "Chromium";v="148"
9 Content-Type: application/json
10 sec-ch-ua-mobile: ?0
11 Origin: http://localhost:3000
12 Sec-Fetch-Site: same-origin
13 Sec-Fetch-Mode: cors
14 Sec-Fetch-Dest: empty
15 Referer: http://localhost:3000/
16 Accept-Encoding: gzip, deflate, br, zstd
17 Accept-Language: en-US,en;q=0.9
18 Cookie: language=en; welcomebanner_status=dismiss; cookieconsent_status=dismiss; csrftoken=kYoKuv3sVe
N9a749ZBXr5c15Ey157b70; sessionid=48j39c4jbgs342dlp5l2symezsysclsje; continueCode=OP86lmrlyzW5LXqG2lt3
cKIkiqqh05cwXT7WSj3hgwfNYAQEBgn7JkwR3xVb; token=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdGF0dXMiOiJz
dWljZXNziIiwiaWF0YSI6eyJpZCJ6MjUsInVzZXJyZWllIjoiiwiZWlhaWwioiJ0ZXNQGVtYWlsLmNvbSIiInBhc3N3b3JkIjoiod
DI3Y2NiMGVlYThhNzA2YzRjMzRlMTY4OTFmODRLN2IiLCJyb2xlbGljoiYWRtaW4iLCJkZmxleGVub2t1biI6IiIsImxhc3Rmb2dpbk
lwIjoiiMC4wLjAuMCIiInByb2ZpbgVjbWFnZSI6Ii9hc3NldHMvchVibGllL2ltYwdlcY9lCGxvYWRzL2RlZmFlbHRBZG1pb5wbmc
ilCJ0b3RwU2VjcV0iIjoiiwiaXNB3Y3RpdMU0OnRydWUsImNyZWFiZWRRBdCI6IjIwMjYtMDctMTAgMDY6NDU6MTIuOTI3ICswMDow
MCIiInVwZGF0ZWRBdCI6IjIwMjYtMDctMTAgMDY6NDU6MTIuOTI3ICswMDowMCIiImRlbGV0ZWRBdCI6bnVsbnVhdH0sImRhbmQzMzY2NjQ2M30.psv3JMgOB2pRKIK3sZG8R0BenbwttUrmIFWqBJhpSRPJ6SFNKCMKdcGLI2mdkQBec5_brktOJIGMJ88fzLS03PDQ7gEsNkvPnor5wjaioUcmsjresJUiIZFu50nVaREyDOKNEsX8X8PVx5Kfk9Lm9xyQ9gL72gsC6mk9ayKKRsM
19 {
20   "message": "posting review as another user",
    "author": "basil@juice-sh.op"
```

```
201 http://localhost:3000/rest/products/24/reviews Drop Forward

Response Pretty Raw +

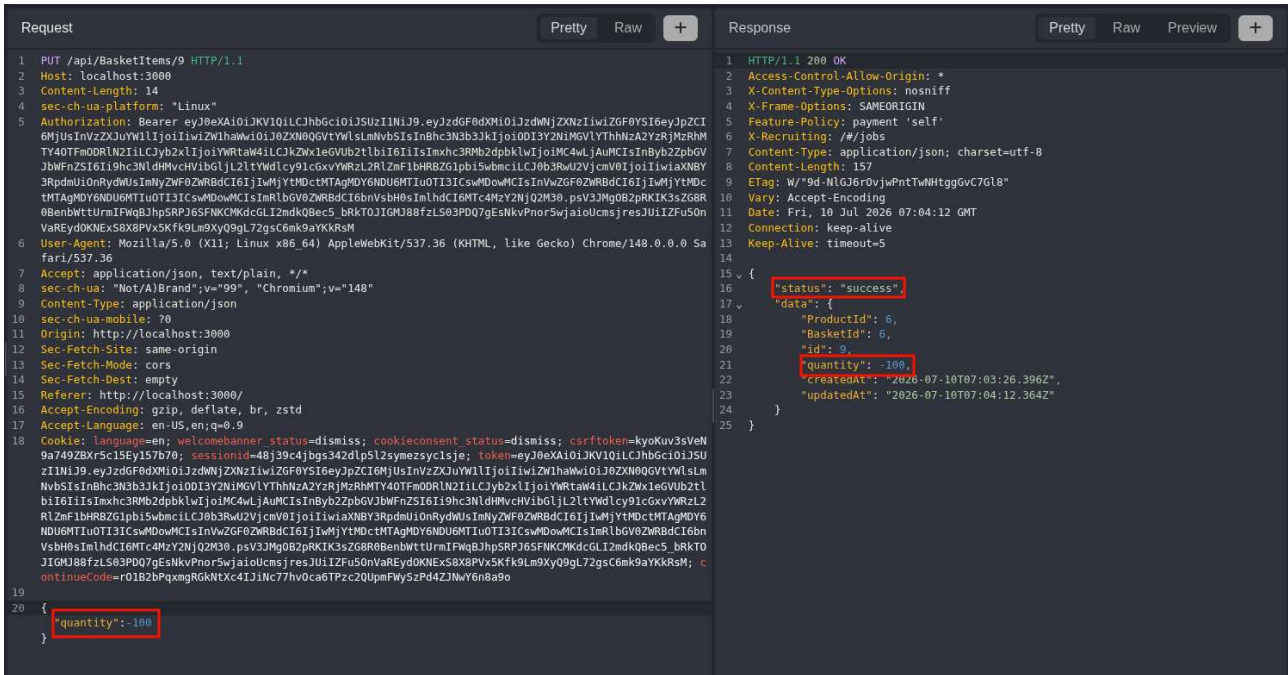
1 HTTP/1.1 201 Created
2 Access-Control-Allow-Origin: *
3 X-Content-Type-Options: nosniff
4 X-Frame-Options: SAMEORIGIN
5 Feature-Policy: payment 'self'
6 X-Recruiting: /#/jobs
7 Content-Type: application/json; charset=utf-8
8 Content-Length: 20
9 ETag: W/"14-Y53wuE/mmbSikKcT/WualL1N65U"
10 Vary: Accept-Encoding
11 Date: Fri, 10 Jul 2026 06:56:27 GMT
12 Connection: keep-alive
13 Keep-Alive: timeout=5
14
15 {
  "status": "success"
}
```

9. Business Logic Flaw / Integer Underflow via Negative Order Quantity - High

CWE	CWE-20 - Improper Input Validation
CVSS 3.1	7.1 / CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:N/I:H/A:L
Owasp Category	A05:2025 - Injection
Root Cause	The application's shopping cart and checkout processing system fails to perform sufficient server-side mathematical validation checks on incoming order quantities. While the frontend user interface prevents a user from selecting a value less than 1 via form boundaries, the backend API maps input variables natively without validating that numeric attributes are strictly positive integers. By capturing the checkout transaction via an intercepting proxy web tool, an attacker can manually alter the <code>quantity</code> parameter value to a negative integer (e.g., <code>-5</code>). Because the backend blindly processes this negative quantity value, it performs inverse multiplication calculations against the product price, reducing the total transaction basket cost or generating an unnatural reverse account balance credit.
Impact	Exploiting integer validation boundaries within checkout flows introduces severe financial logic disruption vectors. An attacker can leverage this flaw to: • Arbitrarily reduce the total cost of an active shopping basket to zero or a minimal value by balancing high-value goods against a massive negative value item. • Steal inventory data records or manipulate financial transaction logic pipelines. • Induce internal database runtime exceptions or arithmetic calculation failures, altering user wallet states or invoice ledgers.
Remediation	All operational variables arriving from a browser must undergo strict type verification, validation matching, and logical assertion checks directly within the server runtime boundary. • Enforce Strict Positivity Validation Checks: Implement validation blocks enforcing strict lower-bound conditions for any parameter that acts as a multiplier within accounting, data modeling, or volume loops. • Implement Sanitization Framework Interceptors: Reject transactions containing out-of-bounds parameters instantly by interrupting the controller processing flow and rendering an explicit HTTP <code>400 Bad Request</code> or transaction error response structure.
References	• OWASP Top 10 Mapping: A05:2025 - Injection (Improper parameter processing and missing input validation boundaries fall under general injection and processing validation errors) [https://owasp.org/Top10/2025/A05_2025-Injection/] • OWASP Cheat Sheet Series: Input Validation Cheat Sheet [https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html]

- MITRE CWE: CWE-20: Improper Input Validation [<https://cwe.mitre.org/data/definitions/20.html>]

Finding Evidence



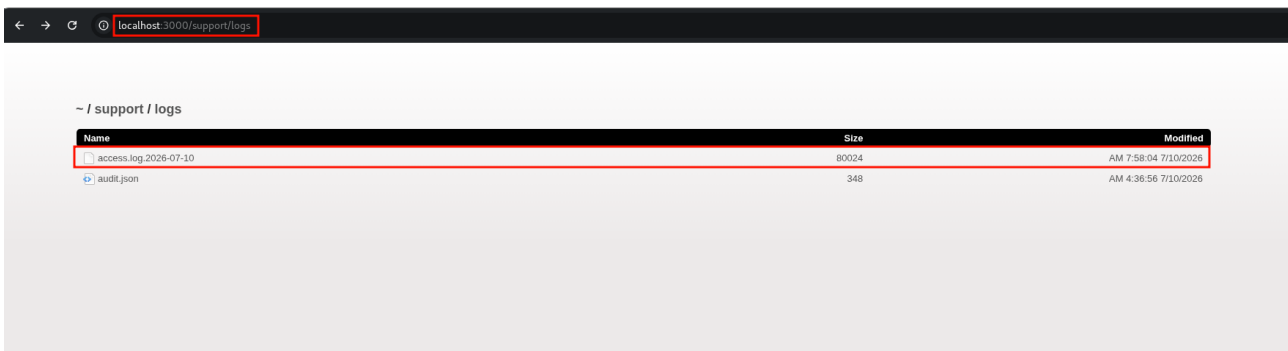
The screenshot displays the Network tab of a web browser's developer tools. The selected request is a PUT to `/api/BasketItems/9` with an HTTP status of 200 OK. The request body is a JSON object: `{ "quantity": -100 }`. The response body is a JSON object: `{ "status": "success", "data": { "productId": 6, "basketId": 6, "id": 9, "quantity": -100, "createdAt": "2026-07-10T07:03:26.396Z", "updatedAt": "2026-07-10T07:04:12.364Z" } }`. The response body is highlighted in red.

10. Sensitive Information Disclosure via Exposed Server Access Log File - High

CWE	CWE-538 - Insertion of Sensitive Information into Externally-Accessible File or Directory
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Owasp Category	A01:2025 - Broken Access Control
Root Cause	An automated directory-busting or content-enumeration scan revealed an exposed server infrastructure directory path (<code>/support/logs</code>). This storage path contains active production transaction logging entities, specifically an <code>access.log</code> text resource. Because the backend web application engine or hosting proxy routing layout fails to restrict read permissions on this administrative path, unauthenticated external network clients can view the resource directly. The <code>access.log</code> file systematically documents incoming client transactions, recording sensitive request elements including cleartext session tokens, resource traversal history, system metadata strings, and sensitive payload fragments cached inside URL parameter chains
Impact	Allowing unrestricted access to base infrastructure log files facilitates catastrophic visibility into ongoing application transactions. An unauthenticated attacker can exploit this data leak to: • Harvest valid Session Identifiers or Authorization Bearer tokens, allowing immediate hijacking of active user accounts. • Map out backend application APIs and routing structures, revealing administrative control endpoints that might otherwise be unlinked or obfuscated. • Trace specific transaction habits, IP mapping endpoints, or systemic configuration signatures, providing highly actionable ammunition for compounding infrastructure exploitation chains.
Affected Component	<code>/support/logs</code>
Remediation	System monitoring artifacts and access tracking logs are proprietary technical infrastructure variables that must never sit inside the public-facing deployment root directories. • Relocate Log Storage Roots Outside Web Roots: Ensure that application log compilation engines write metadata streams directly into a secure system folder framework located completely outside the active public web server rendering path (e.g., <code>/var/log/app/</code> instead of <code>/public/support/logs/</code>). • Configure Restricted Endpoint Ingress Bounds: Apply routing blocks inside your reverse proxy or web engine to explicitly match, trap, and deny external traffic targeted at configuration extensions (<code>.log</code>, <code>.ini</code>, <code>.conf</code>). • Enforce Strict Privilege Contexts: If logs must be accessible over an HTTP interface for diagnostic purposes, isolate the path behind standard

	administrative authentication sessions and implement rigid source IP range restriction access patterns.
References	• OWASP Top 10 Mapping: A01:2025 - Broken Access Control (Unrestricted access to high-severity structural diagnostics and sensitive administration file segments) [https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/] • OWASP Cheat Sheet Series: Logging and Monitoring Cheat Sheet [https://cheatsheetseries.owasp.org/] • MITRE CWE: CWE-538: Insertion of Sensitive Information into Log File [https://cwe.mitre.org/data/definitions/538.html]

Finding Evidence



11. Insecure Direct Object Reference (IDOR) / Broken Object Level Authorization (BOLA) in Basket Access - Medium

CWE	CWE-639 - Authorization Bypass Through User-Controlled Key
CVSS 3.1	6.5 / CVSS:3.1/AV:N/AC:L/PR:L/UI:N/S:U/C:H/I:N/A:N
Owasp Category	A01:2025 - Broken Access Control
Root Cause	The application's shopping basket retrieval functionality suffers from an Insecure Direct Object Reference (IDOR) vulnerability, also categorized under Broken Object Level Authorization (BOLA). When a user navigates to their shopping cart, the client issues a <code>GET</code> request to <code>/rest/basket/[id]</code>, where <code>[id]</code> represents the unique identifier of the target basket. However, the backend server fails to validate whether the user initiating the request owns or is authorized to view the requested basket resource. The system maps the <code>id</code> path parameter directly to a database query and returns the records without checking if the basket's owner identity matches the identity bound to the user's active session token. By changing the numerical index from their own basket ID (e.g., <code>5</code>) to any other integer, an authenticated attacker can view the basket contents of any user on the platform.
Impact	Unauthorized access to active user shopping baskets exposes sensitive user data and shopping behaviors. An attacker can exploit this flaw to: • Exfiltrate Personally Identifiable Information (PII) or business data indirectly linked via basket metrics (e.g., specific high-value items, custom configurations, quantities, or pricing discounts applied to other accounts). • Map out active user volumes and enumerate sequential internal transaction records. • Facilitate further targeted social engineering or phishing attacks by monitoring what products a corporate client or individual user is currently intending to purchase.
Remediation	An application must always enforce object-level access verification boundaries on the server side using the session identity context rather than trusting client-supplied object identifiers. • Enforce Strict Object-Level Authorization Checks: Before returning a database model to the requester, execute an authorization check verifying that the <code>UserId</code> associated with the requested <code>Basket</code> record matches the authenticated user ID claims parsed securely from the active request token. • Use Indirect or Opaque References: Instead of exposing raw, predictable sequential database integer keys (<code>5</code>, <code>6</code>, <code>7</code>) in public API URLs, map identifiers using cryptographically secure, non-enumerable alternatives such as random UUIDv4 values.
References	• OWASP Top 10 Mapping: A01:2025 - Broken Access Control (Flaws categorized under broken object-level authorization and horizontal privilege escalation) [https://owasp.org/Top10/2025/A01_2025-Broken_Access_Control/]

- OWASP Cheat Sheet Series: Authorization Cheat Sheet [https://cheatsheetseries.owasp.org/cheatsheets/Authorization_Cheat_Sheet.html]
- MITRE CWE: CWE-639: Authorization Bypass Through User-Controlled Key [<https://cwe.mitre.org/data/definitions/639.html>]

Finding Evidence

[illegible]

12. Arbitrary File Download / Access Control Bypass via Null Byte Injection - High

CWE	CWE-159 - Improper Handling of Invalid Use of Special Elements
CVSS 3.1	7.5 / CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:N/A:N
Owasp Category	A05:2025 - Injection
Root Cause	The application features a file retrieval route under the <code>/ftp/</code> directory designed to distribute specific public assets. To prevent users from downloading unauthorized system files, backup archives, or source code, the application enforces a strict extension restriction on the frontend and backend routing logic, nominally permitting only files ending in <code>.md</code> or <code>.pdf</code>. However, the file validation mechanism is vulnerable to a URL-encoded Null Byte Injection flaw (<code>%00</code> or double-encoded <code>%2500</code>). When an attacker requests a restricted file extension appended with an encoded null byte and an allowed trailing extension (e.g., <code>coupons_2013.md.bak%2500.md</code>), the application's high-level routing logic validates the file by looking at the very end of the string, noticing the acceptable <code>.md</code> extension. When the string is passed down to lower-level filesystem APIs (such as C/C++ or older runtime file-reading libraries), the string is treated as a null-terminated sequence. The filesystem layer truncates the path right at the null character, dropping the trailing <code>.md</code> entirely and reading the forbidden asset (<code>coupons_2013.md.bak</code>) instead.
Impact	Bypassing file extension restrictions allows unauthenticated attackers to fetch restricted or hidden files from the hosting repository. An attacker can exploit this flaw to: • Exfiltrate legacy database backups, configuration files, or transaction logs containing sensitive corporate parameters. • Discover sensitive developer notes, testing files, or code artifacts (<code>.bak</code>, <code>.old</code>, <code>.src</code>) containing secondary architectural vulnerabilities. • Enumerate the internal directory layout by probing for common hidden files.
Affected Component	<code>/ftp/[file_path]</code>
Remediation	Applications must perform input validation using runtime utilities that are immune to string-termination exploits, ensuring that parameters are thoroughly sanitized before hitting native API handlers. • Sanitize Inputs for Null Characters: Implement a global request interceptor or middleware validation layer that scans all dynamic file access inputs for the presence of null bytes (<code>\0</code>, <code>%00</code>, or <code>%2500</code>) and outright rejects the request with an HTTP <code>400 Bad Request</code>.

	• Utilize Secure Filesystem Operations: Ensure that file paths are evaluated using modern string handling and filesystem API modules that ignore null-termination mechanics or inherently reject raw path concatenation. • Employ White-List File Indexing: Avoid passing user-controlled file paths or extension matrices to native storage frameworks. Instead, map downloadable assets to static random identifiers (such as a database index or UUIDv4) and fetch files internally using that safe token dictionary.
References	• OWASP Top 10 Mapping: A05:2025 - Injection (Improper neutralization of specialized data formats leading to path and execution modification) [https://owasp.org/Top10/2025/A05_2025-Injection/] • OWASP Cheat Sheet Series: Input Validation Cheat Sheet [https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html] • MITRE CWE: CWE-159: Failure to Sanitize Null Elements [https://cwe.mitre.org/data/definitions/159.html]

Finding Evidence



A Appendix

A.1 External Security References

The following authoritative reference materials provide formal documentation, vulnerability taxonomy, and baseline remediation guidelines for the findings discovered during this engagement:

- **OWASP Top 10 API Security Project:** [A01:2023 - Broken Object Level Authorization](#)
- **OWASP Vulnerability Guidance:** [Injection Flaws & Parameter Sanitization](#)
- **MITRE CWE Database:** [CWE-89: Improper Neutralization of Special Elements used in an SQL Command](#)
- **MITRE CWE Database:** [CWE-159: Failure to Sanitize Null Elements](#)
- **PortSwigger Web Security Academy:** [Cross-Site Scripting \(XSS\) Cheat Sheet](#)

End of Report